The Interference Budget

How a memory that never grows still forgets — and why that is the whole story of linear attention and BDH

DataForge 2026 · Pathway Track · companion blog to the interactive explainer

---

One sentence, and a dare

Here is the sentence the explainer teaches:

> *A fixed-size fast-weight memory* $S = \Sigma\ k_i v_i^\top$ *stores a stream of key→value bindings without ever growing, but because every binding is added into the same matrix, reading back an earlier value returns the true value plus a cross-talk term* $\Sigma_{j \neq i}\ (k_i \cdot k_j) v_j$ *whose size is set by how much the keys overlap — so recall error grows like* $\sqrt{(N/d)}$ *and is near-zero only when the keys are near-orthogonal.*

It is falsifiable. If you can keep recall sharp while packing many *correlated* keys into the memory, the sentence is wrong. The interactive artifact gives you every knob you need to try. This post explains where the sentence comes from, why each clause is true, and why it is the right lens for understanding Pathway's Dragon Hatchling (BDH).

---

1. Two ways to remember the past

Softmax attention remembers by *keeping*. Every token leaves a `(key, value)` pair in a cache; answering a query means comparing the query to every stored key. The cache grows with sequence length, and so do the memory footprint and the per-token compute. This is the well-known cost of long context, and the reason a small industry exists around KV-cache eviction, compression and retrieval.

A **fast-weight** memory remembers by *folding*. It never stores individual pairs. It keeps one matrix S and, for each binding (k_i, v_i), adds the rank-1 outer product into it:

$$S_i = S_{i-1} + k_i\ v_i^\top \qquad (S \text{ is } d \times m, \text{ fixed size, independent of } i)$$

This is a Hebbian rule: patterns that are active together get their connection strengthened. Schmidhuber proposed exactly this "fast weight" scheme in 1992; Ba, Hinton and colleagues revived it in 2016. In the explainer's §1 you can step through twenty writes and watch S fill in — after twenty it looks like noise, but every binding is still in there, folded rather than filed.

To read the value for a key q:

$$\text{out} = S^\top q = \Sigma_j\ (k_j \cdot q)\ v_j$$

Constant memory, linear-time inference. The catch is in the next section.

---

2. What you get back is signal plus cross-talk

Set $q = k_i$, one of the keys you stored, and split the sum. With unit-norm keys $k_i \cdot k_i = 1$:

```
out = v_i  +  Σ_{j≠i} (k_i · k_j) v_j
      ────    ──────────────────
      signal   cross-talk
```

The cross-talk is not noise the system injected. It is *the other stored values, leaking in through the dot products between their keys and yours*. If every other key is exactly perpendicular to k_i, every $k_i \cdot k_j = 0$ and recall is exact. As keys tilt toward each other, their values bleed into your answer.

The explainer's §2 draws the true v_i and the returned $S^T k_i$ as side-by-side bars, plus the analytic cross-talk vector. The gap between the bars *is* the cross-talk vector — we are not estimating the error, we are decomposing it.

A common misconception, checked. "A bigger state means a longer memory." A $d \times m$ state has only $\min(d, m)$ independent directions. Beyond that, new writes must reuse directions already spoken for, and interference is forced by linear algebra, not by a design choice you can tune away.

---

3. The law: $\sqrt{(N/d)}$

The cross-talk term is a sum of $N-1$ roughly independent random vectors, each scaled by a dot product $k_i \cdot k_j$. For random unit keys in d dimensions, $E[(k_i \cdot k_j)^2] \approx 1/d$. Values have unit norm. So the expected squared error is about $(N-1)/d$, and:

$$\text{RMS recall error} \approx \sqrt{(N-1) \cdot (\rho^2 + (1-\rho^2)/d)} \to \approx \sqrt{(N/d)} \text{ when } \rho = 0$$

where ρ is the average pairwise key correlation. The explainer's §3 sweeps N or d, runs dozens of real trials per point, and plots the measured RMS error against this predicted curve. At $\rho = 0$ the dots sit on the line (mean absolute deviation ≈ 0.03). Raise ρ and the measured curve lifts off: the ρ^2 term inside the square root becomes a floor that no amount of extra d can remove. That gap between the two curves is the entire lesson in one picture.

This is the same capacity story that associative-memory theory has told for decades. Classical Hopfield networks store about $0.14 \cdot d$ patterns before recall breaks (Hopfield, 1982). Modern Hopfield networks — which turn out to be mathematically equivalent to attention — push that capacity up exponentially by making the read *nonlinear* (Ramsauer et al., 2021). Our linear memory sits at the conservative end of that spectrum, and now you can measure exactly where.

You do not have to trust the browser. The repository has a ten-line NumPy function that reproduces the curve; `rms_recall_error(30, 40)` ≈ $\sqrt{(29/40)}$.

---

4. Interference is not the same as decay

There are two different ways a fixed state loses information, and conflating them is a frequent error.

Mechanism	Update	Effect	Cost
Interference	$S_i = S_{i-1} + k_i v_i^T$	all bindings kept with equal weight; they blur as they pile up	old facts buried, never removed; error grows with N
Decay (forget gate)	$S_i = \lambda \cdot S_{i-1} + k_i v_i^T$, $\lambda < 1$	down-weights the past each step; recent bindings dominate	distant facts genuinely gone, whether or not they still matter

Gated linear attention, RetNet, Mamba's state updates and DeltaNet all live in this table — they differ mainly in *how* λ and the write are computed (scalar, diagonal, data-dependent, or a rank-1 correction). The explainer's §4 plots recall error against binding age: at $\lambda = 1$ the curve is flat (interference hits every binding about equally); drop λ and it tilts (recent sharp, old gone). Neither is "better." A model answering questions about the start of a long document wants λ near 1 and pays in interference; a model tracking a fast-changing state wants low λ and pays in amnesia.

The **delta rule** (DeltaNet, Yang et al., 2024) is the third option in the explainer: before writing binding i, subtract the memory's current prediction for k_i, so you write only the correction $v_i - S_{i-1}{}^{\top} k_i$. This removes interference for repeated or highly similar keys, at the cost of a slightly more complex, less trivially parallel update.

---

5. This *is* linear attention

Take softmax attention and replace $\exp(q^{\top}k)$ with a plain feature-map dot product $\phi(q)^{\top}\phi(k)$ (Katharopoulos et al., 2020):

$$
\begin{aligned}
\text{linear:}\quad \mathrm{out}_i &= \Sigma_{j \le i}\ \big(\ \phi(q_i)\cdot\phi(k_j)\ \big)\ v_j \\
&= \phi(q_i)^{\top}\ \big(\ \Sigma_{j \le i}\ \phi(k_j)\ v_j{}^{\top}\ \big) \\
&= \phi(q_i)^{\top}\ S_i
\end{aligned}
$$

The parenthesised term $S_i = \Sigma_{j \le i}\ \phi(k_j) v_j{}^{\top}$ is our memory, and it obeys $S_i = S_{i-1} + \phi(k_i) v_i{}^{\top}$ — the exact Hebbian write from §1. So "linear attention," "a fast-weight matrix," and "a Hebbian associative memory" are three names for one object. Moving from softmax to linear is precisely moving from *a cache you scan* to *a state you accumulate*.

The explainer's §5 puts six bindings under a softmax read and a linear read side by side. Softmax can concentrate almost all its weight on the true match; the linear read leaves a tail on the other bindings — that tail is the cross-talk. Switch the feature map ϕ to relu and the contributions become non-negative and sparse: most bindings contribute exactly zero. That is the doorway to BDH.

---

6. Where this lives: Dragon Hatchling (BDH) and BDH-CQ

Pathway's Dragon Hatchling is a Post-Transformer architecture whose stated design move is to *reformulate attention as synaptic memory that updates as the model reads* (Kosowski et al., 2025). The concept in this explainer is not an analogy for BDH's mechanism — the fast-weight / linear-attention state **is** the mechanism, in BDH's GPU-friendly form.

The shared skeleton. The Dragon Hatchling paper (arXiv:2509.26507, §6.1) defines BDH-GPU as "a combination of two blocks: a specific kind of *ReLU-lowrank* feed-forward network, and a *linear attention* mechanism." Its positive activations are sparse at "about 5% level" (§6.4). The outer-product write in §1 is that linear-attention / fast-weight rule; BDH makes it trainable and scalable, and matches GPT-2 on language and translation at equal parameters from 10M to 1B (§4.2).

What BDH adds, with evidence labels:

Ingredient	Detail	Evidence level
ReLU-lowrank feed-forward + linear attention	the GPU-friendly form of the neuron–synapse dynamics — the write/read you explored (§6.1)	**formal, in paper**
GPT-2 parity at 10M–1B params, language + translation (§4.2)	Transformer-like scaling laws	**formal, in paper**
Positive activations sparse at ~5% (§6.4)	keeps "keys" concept-selective, so writes stay legible and interference stays low	developer-reported
Monosemanticity on language tasks; scale-free graph, heavy-tailed degree distribution	specific synapses strengthen for specific concepts; a few hub neurons carry many connections	developer-reported interpretability results
Sudoku Extreme: **97.4%** over ~250k puzzles, no chain of thought; leading LLMs ≈ 0%	constraint-reasoning evidence the evolving state supports multi-step inference	developer-reported (Pathway research post, Mar 2026)
Pretraining scaling laws reported to ~600B params; developed on Amazon SageMaker HyperPod	trains predictably at scale on standard infrastructure — largest *benchmarked* model in the family is 150M	developer-reported (Pathway / AWS)

BDH-CQ: the additive-per-demonstration case. BDH-CQ (arXiv:2608.09888) "combines in-context learning with recurrent latent reasoning … without verbalizing its intermediate reasoning." Its state update is $S_t = U\theta(S_{t-1}, D_t)$ for the t-th demonstration D_t, and its report calls the interpretation "generally related to attention, fast-weight memory, and linear-attention views of contextual association … with linear attention being the conceptually simplest standalone realization … capturing the special case $S_t = S_{t-1} + U\theta(D_t)$" (§3.2) — exactly $S = \Sigma\, k_i v_i^\top$ with i indexing demonstrations. Two consequences follow directly:

- **Adaptation without weight updates.** "Neither task identifiers nor evaluation-task demonstration pairs participate in training, and no parameters are updated at inference time." The "learning" is the accumulation of outer products into S during the forward pass — weights frozen, the *state* carries the new rule. A 150M-parameter BDH-CQ reaches **29.5% pass@2 on public ARC-AGI-1** (400 tasks) at ~**$0.0007 per task**. Contrast HRM / TRM, whose "ARC pipeline is transductive: demonstration pairs from evaluation tasks are augmented and used in optimization … a previously unseen hidden task therefore requires backward-pass adaptation before it can be evaluated" (§8).
- **Effort is recurrent compute, not tokens.** BDH-CQ is trained with low / medium / high latent-reasoning levels, selectable at inference — pass@2 of **21.0 / 27.0 / 29.5%** as effort rises. The extra cost is more iterations over the state, not more chain-of-thought text. Demonstration coverage is then an interference-budget question: more demonstrations sharpen the recalled rule until their overlap starts blurring it — the trade-off from §3.

Where BDH-CQ has no direct role. The $\sqrt{N/d}$ capacity law and the softmax-vs-linear comparison are properties of linear attention in general. BDH-CQ neither introduced nor depends on them; it inherits them by building on the family. Saying otherwise would be inventing a connection.

Evidence discipline. *Formal, in paper* means an equation or definition stated in the arXiv papers (the ReLU-lowrank + linear-attention formulation; 10M–1B GPT-2 parity). *Developer-reported* means a result Pathway or AWS published — the ~5% sparsity measurement, Sudoku 97.4%, ARC-AGI 29.5%, the 600B scaling figure, HyperPod development — with no independent reproduction. A benchmark score is not a deployment. The toy in the explainer is an illustration — a hand-built memory, not a BDH checkpoint. What *is* solid: the linear-attention and associative-memory results in §§1–5 are standard, published and widely replicated.

---

7. Limitations, stated plainly

1. **The toy uses near-ideal keys.** We hand it random or deliberately correlated unit vectors. A real model must *learn* to place near-orthogonal keys, and often does not — so real linear-attention models forget more than the $\sqrt{}$ law's best case.
2. **Linear attention is strictly less expressive than softmax + full cache** for exact retrieval. A growing KV cache can return any past value exactly; a fixed state cannot, once you exceed its rank. Benchmarks that reward precise long-range copying expose this.
3. **Within-session memory is not learning.** Everything here happens in S during one forward pass. Close the session and it is gone. Consolidating useful fast state into durable slow weights is an open problem the BDH authors name explicitly.
4. **The $\sqrt{}$ law is an average.** It assumes random keys and unit values. Structured or adversarial keys, or heavy-tailed value norms, break it in both directions.
5. **Illustration vs computation.** §§1–5 of the explainer are live arithmetic in your browser. §6's BDH equations are transcribed and simplified from the papers; the neuron–synapse framing is a teaching schematic, not a trace of a real BDH run.

References

- J. Schmidhuber (1992). *Learning to control fast-weight memories.* Neural Computation 4(1).
- J. Ba, G. Hinton, V. Mnih, J. Leibo, C. Ionescu (2016). *Using Fast Weights to Attend to the Recent Past.* NeurIPS 2016.
- J. Hopfield (1982). *Neural networks and physical systems with emergent collective computational abilities.* PNAS 79(8).
- H. Ramsauer et al. (2021). *Hopfield Networks is All You Need.* ICLR 2021.
- A. Katharopoulos, A. Vyas, N. Pappas, F. Fleuret (2020). *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention.* ICML 2020.
- S. Yang, B. Wang, Y. Shen, R. Panda, Y. Kim (2024). *Gated Linear Attention Transformers with Hardware-Efficient Training*. ICML 2024.
- S. Yang, B. Wang, Y. Zhang, Y. Shen, Y. Kim (2024). *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. NeurIPS 2024.
- A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, M. Bartoszkiewicz (2025). *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain.* Pathway. arXiv:2509.26507 (30 Sep 2025). Companion code: github.com/pathwaycom/bdh.
- B. Engdahl, A. Kosowski, J. Chorowski, Z. Stamirowska, P. Uznański, et al. (2026). *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*. Pathway. arXiv:2608.09888 (10 Aug 2026).
- Pathway (17 Mar 2026). *Beyond Transformers: solving Sudoku Extreme.* pathway.com/research/beyond-transformers-sudoku-bench.
- AWS Startups. *Pathway's BDH: a new post-transformer approach to enterprise AI, on AWS.* aws.amazon.com — SageMaker HyperPod development; ~600B pretraining-scaling figure.
- HRM / TRM contrast: drawn in BDH-CQ §8 ("Task-trained recursive solvers").